Decentralized Electronic Health Records using Hyperledger Fabric & IPFS

A Blockchain-Based EHR with Consent Management and Aggregate Signature Verification

Submitted by:

Avijit Ram – 25CSM2R05
Jenish Shiroya – 25CSM2R10
Rahul Yadav – 25CSM2R16

Under the guidance of

Prof. E. Suresh Babu

Department of Computer Science and Engineering
National Institute of Technology, Warangal

April 24, 2026

Abstract

This comprehensive report presents the design, implementation, and deployment of a Decentralized Electronic Health Records (EHR) system built on Hyperledger Fabric 2.5 blockchain technology combined with IPFS via Kubo for off-chain storage. The system provides a secure, transparent, and patient-empowering platform for health record management with cryptographically enforced role-based access control across three organizations (Hospital, Diagnostics, and Insurance Provider), five peers, six backend microservices, and two React frontend portals deployed across three physical machines. The two core novel contributions are: a cryptographically secure consent management system where role identity is embedded directly in X.509 digital certificates and enforced at the chaincode layer making access control unforgeable by design, and an aggregate transaction verification mechanism that distills an entire blockchain transaction history into a single SHA-256 hash giving patients a simple verifiable integrity proof of their records. A third contribution covers OCR-assisted medical record ingestion with IPFS archival and client-side EHR PDF export.

Contents

1 Introduction

1.1 Project Overview

Traditional Electronic Health Record (EHR) systems are centralized, siloed, and patient-disempowering. They lock medical data within a single hospital's database, offer no patient control over data access, provide no tamper-evident audit trail, and face serious scalability bottlenecks when storing large clinical payloads on-chain. This project addresses all four problems through a hybrid architecture combining Hyperledger Fabric 2.5 (a permissioned blockchain) with IPFS via Kubo (decentralized off-chain storage), resulting in a fully functional, multi-organization, multi-peer decentralized EHR system.

1.2 Objectives

- Implement a cryptographically secure consent management system using role attributes embedded in Fabric CA X.509 certificates
- Provide peer-wise role-based access control enforced at three layers: API, JWT, and chaincode
- Ensure data integrity through hybrid on-chain (Hyperledger Fabric) and off-chain (IPFS) storage with CID-based tamper detection
- Deploy a fully distributed network across three physical machines with three organizations and five peers
- Implement aggregate transaction verification giving patients a single verifiable integrity hash over their complete record history
- Build intuitive web portals for hospital staff and patients with full consent management UI
- Integrate OCR-assisted ingestion of paper medical documents into the blockchain-backed EHR

1.3 Key Features

- **Dual Portal System:** Separate hospital portal (for staff) and patient portal (for patients) with role-aware routing
- **Cryptographic RBAC:** Role attributes embedded in X.509 certificates via Fabric CA – enforced by chaincode, not just application logic
- **Hybrid Storage:** Small structured data on-chain; large clinical JSON documents in IPFS with CID anchoring on-chain
- **Patient Consent Management:** Section-level access grants with expiry, instant revocation, access request/approval workflow, and immutable audit log
- **Aggregate Signature Verification:** SHA-256 digest of sorted blockchain txIds giving patients a single verifiable integrity fingerprint
- **OCR Medical Ingestion:** Tesseract.js pipeline with IPFS archival before recognition, structured attribute extraction, and two-stage UX

- **Full Visit Lifecycle:** 11-state visit status machine from OPEN to DISCHARGED with immutable forwarding log
- **Insurance Claims Pipeline:** Submit, audit, and process insurance claims with on-chain claim metadata
- **Admin-Controlled Staff Registration:** Atomic dual-store registration combining bcrypt credential store with Fabric CA enrollment and rollback on failure

2 Problem Statement

Health data today faces four structural problems that no centralized EHR solution has satisfactorily resolved:

Centralized data silos. Patient records are locked within a single institution. A patient visiting a different hospital carries no verifiable record of their history. Inter-hospital sharing requires bilateral agreements, custom integrations, and still produces unverified copies.

Lack of patient agency. In a traditional EHR system, the hospital is the custodian. The patient has no mechanism to know who accessed their data, to restrict access to specific sections, or to revoke access once granted. Consent is either global or nonexistent.

Absence of a tamper-evident audit trail. Traditional relational databases can be modified by a database administrator. There is no cryptographic proof that a diagnosis entered on a particular date was not subsequently changed. In medicolegal disputes, this is a fundamental gap.

Scalability bottlenecks from on-chain storage. Storing full clinical payloads directly on a blockchain ledger causes rapid ledger bloat, making the system impractical at scale.

3 System Architecture

3.1 Overall Architecture

The system follows a four-tier hybrid architecture:

1. **Presentation Layer:** React + Vite + Tailwind CSS portals (hospital portal, patient portal)
2. **Application Layer:** Six Node.js Express microservices (peer0-api, peer1-api, peer2-api, extorg-api, patient-api, ipfs-service)
3. **Blockchain Layer:** Hyperledger Fabric 2.5 with eight smart contracts on `ehrchannel`
4. **Off-chain Storage:** IPFS via Kubo for large clinical JSON documents; SQLite for patient application credentials only

3.2 Technology Stack

3.2.1 Frontend Technologies

- React 18 + Vite: Component-based UI development with fast HMR
- Tailwind CSS: Utility-first CSS framework

- React Router v6: Client-side routing
- Axios: HTTP client for API communication
- lucide-react: Icon library for UI components
- jsPDF + jspdf-autotable: Client-side EHR PDF generation

3.2.2 Backend Technologies

- Node.js + Express.js: Runtime and web framework for all six microservices
- Hyperledger Fabric 2.5: Enterprise permissioned blockchain platform
- @hyperledger/fabric-gateway: Client SDK for Gateway API
- @grpc/grpc-js: gRPC transport for peer communication
- jsonwebtoken + bcryptjs: JWT session management and password hashing
- express-validator: Input validation on all API routes
- better-sqlite3 (WAL mode): Patient application credential store
- Winston: Structured logging across all services

3.2.3 Supporting Technologies

- IPFS via Kubo: Decentralized content-addressed file storage
- Tesseract.js: OCR engine for medical document digitization
- Fabric CA: Certificate Authority for identity enrollment with role attributes
- Docker + Docker Compose: Containerized network deployment (per-machine)
- Node.js crypto module: SHA-256 for aggregate transaction hash computation

3.3 Network Architecture

The Hyperledger Fabric network consists of three organizations representing different healthcare ecosystem actors:

Organization	MSP ID	Role	Peers
Hospital	HospitalMSP	Primary care provider	peer0 (:7051), peer1 (:9051), peer2 (:10051)
Diagnostics	DiagnosticsMSP	Lab & radiology	peer0.diagnostic (:8051)
Provider / Insurance	ProviderMSP	Billing, claims, insurance	peer0.provider (:11051)

All five peers joined a single Fabric channel named `ehrchannel`. All three organizations hold a full replica of the ledger. Ordering is handled by a single Raft CFT orderer node (`orderer.example.com:7050`). The three organizations were physically distributed

across three machines (SYS1, SYS2, SYS3) operated by different subgroups. The Hospital Organization subgroup (SYS1) hosted peer0, peer1, peer2, the orderer, all four Fabric CAs, and the IPFS node.

4 Data Design – On-Chain and Off-Chain Split

4.1 Ledger Key Scheme

The blockchain stores only small, structured anchor records under deterministic namespaced keys:

```
PATIENT:<patientId>   -- Demographics, visitIds, ehrCID, visitCount
EHR:<patientId>       -- currentCID, cidHistory[], createdAt, updatedAt
VISIT:<visitId>       -- status, assignedDoctor/Nurse, visitCID,
   cidHistory[], claimFields
ACCESS:<patientId>    -- grants[], requests[], auditLog[]
```

Listing 1: Ledger Key Prefixes

4.2 On-Chain vs IPFS Boundary

The core architectural insight: separate what needs to be *verifiable and immutable* from what needs to be *stored*. A CID (Content Identifier) is the SHA-256 hash of the IPFS file's content. Any tampering changes the CID, invalidating the on-chain reference.

Data	Location	Reason
Patient demographics	Both (on-chain + EHR JSON)	On-chain for fast lookup; IPFS for rich context
EHR current CID + cidHistory	On-chain	Integrity anchor – CID is tamper-evident
Actual EHR JSON (allergies, conditions, etc.)	IPFS	Large, variable-length; avoids ledger bloat
Visit status, assignedDoctor, claimStatus	On-chain	Small fields, needed for chaincode logic
Visit clinical content (vitals, notes, prescriptions)	IPFS	Large payload; fetched on demand
Access grants, audit log	On-chain	Critical for verifiable consent; must be in ledger
Claim ID, amount, decision	On-chain	Insurance auditors need tamper-proof records

4.3 Two-Template IPFS Design

Two distinct JSON templates are used, reflecting a fundamental ownership distinction:

EHR Document (Patient-Owned): Lifelong record persisting across all visits. Contains `allergies[]`, `chronicConditions[]`, `ongoingMedications[]`, `surgicalHistory[]`,

`familyHistory[]`, `immunizations[]`, `emergencyContact`, `demographics`, and `lifestyle`. Only doctors and nurses with explicit patient consent grants may write to it.

Visit Document (Hospital-Owned): Single encounter record, closed at discharge. Contains `chiefComplaint`, `forwardingLog[]`, `diagnosisNotes`, `finalDiagnosis`, `prescriptions[]`, `vitals`, `careNotes[]`, `labRequests[]`, `medicationDetails`, and `dischargeNotes`. All clinical staff have broad write access during the encounter.

```
{
  "_type": "EHR", "_version": 1, "patientId": "PAT-001",
  "demographics": { "name":"", "age":"", "gender":"", "bloodGroup":"",
     ... },
  "allergies": [],
  // entry: { substance, reaction, severity, addedBy, addedAt }
  "chronicConditions": [],
  "ongoingMedications": [],
  "surgicalHistory": [],
  "familyHistory": [],
  "immunizations": [],
  "emergencyContact": { "name":"", "relation":"", "phone":"", "address":
     "" },
  "lifestyle": { "smoking":"", "alcohol":"", "exercise":"", "diet":"" }
}
```

Listing 2: EHR Template Structure (emptyEHR)

5 Chaincode Architecture – 8 Smart Contracts

The `ehr-chaincode-v3/` directory contains eight smart contracts written in Node.js using the `fabric-contract-api` package. They share a common helper module, `accessControl.js`, which centralizes identity extraction, role enforcement, and ledger utilities.

5.1 accessControl.js – Shared Security Foundation

This is the foundational module that all eight contracts import. It is the technical underpinning of the consent management system.

```
// Extract role directly from CA-signed X.509 certificate -- no DB
   lookup
function getCallerIdentity(ctx) {
  const id = ctx.clientIdentity;
  const role = id.getAttributeValue('role') || '';
  const mspId = id.getMSPID();
  const cnMatch = id.getID().match(/CN=([^,:/]+)/);
  const userId = cnMatch ? cnMatch[1] : 'unknown';
  return { role, mspId, userId };
}

// Assert caller has one of the allowed roles -- throws if not
function requireRole(ctx, ...allowedRoles) {
  const { role, userId } = getCallerIdentity(ctx);
  if (!allowedRoles.includes(role)) {
    throw new Error(
      `Access denied: role '${role}' (user: ${userId}) not permitted.`
    );
```

```
  }
  return { role, userId };
}

// Doctor and nurse require explicit patient consent for EHR section
async function assertReadAccess(ctx, patientId, section) {
  const { role, userId } = getCallerIdentity(ctx);
  if (STAFF_ROLES.includes(role)) {
    if (EHR_CONSENT_REQUIRED_ROLES.includes(role) && section === 'ehr') {
      // fall through to grant check
    } else { return; }  // other staff always pass
  }
  const accessRecord = await getState(ctx, accessKey(patientId));
  const grant = accessRecord?.grants?.find(g =>
    g.granteeId === userId && !g.revoked &&
    (g.expiresAt === '' || g.expiresAt > new Date().toISOString()) &&
    (g.sections.includes('all') || g.sections.includes(section))
  );
  if (!grant) throw new Error(`Access denied: no valid grant for '${userId}'`);
}
```

Listing 3: Core identity and access functions in accessControl.js

5.2 PatientContract

Handles the patient master record stored under `PATIENT:<patientId>`. Key functions:

- **RegisterPatient** – receptionist/admin only; creates patient with demographics, empty `visitIds[]`, and `ehrCID` from IPFS initialization
- **GetPatient** – runs `assertReadAccess` before returning
- **ListAllPatients** – receptionist/admin only; range query over `PATIENT:` prefix
- **UpdatePatientInfo** – updates contact/address only (non-clinical)
- **SetPatientEhrCID** – links on-chain patient record to IPFS EHR document
- **GetPatientHistory** – full blockchain transaction history via `getHistoryForKey`

5.3 VisitContract

The most central contract, managing the 11-state visit lifecycle under `VISIT:<visitId>`:

```
OPEN
 -> WITH_DOCTOR          (AssignDoctor)
     -> WITH_NURSE       (ForwardToNurse)  -- back and forth freely
     -> WITH_LAB         (ForwardToLab)    -- multiple times allowed
     -> VISIT_FINALIZED  (FinalizeVisit -- doctor)
         -> RECORD_FINALIZED  (FinalizeRecord -- medrecordofficer)
             -> CLAIM_SUBMITTED   (SubmitClaim)
                 -> CLAIM_UNDER_AUDIT (AuditClaim)
                     -> CLAIM_APPROVED / CLAIM_REJECTED (ProcessClaim)
                         -> DISCHARGED (DischargePatient)
```

Listing 4: Visit Status State Machine

Key design: `OpenVisit` requires the backend to first pin an empty visit JSON to IPFS and pass the `visitCID` as an argument. All clinical content lives in IPFS; the on-chain record stores only status, assignments, CID pointer, and claim metadata. `UpdateVisitCID` is the central update mechanism called by all backends after modifying IPFS content – it appends the new CID to `cidHistory[]` with actor, timestamp, and reason.

5.4 ForwardContract

Manages role-to-role forwarding events with status transitions:

- **ForwardToNurse** – doctor only; allowed from WITH_DOCTOR, WITH_NURSE, or WITH_LAB; advances to WITH_NURSE
- **ForwardToDoctor** – nurse only; from WITH_NURSE; returns to WITH_DOCTOR
- **ForwardToLab** – doctor only; adds lab request entry to IPFS JSON; advances to WITH_LAB
- **LabResultsBackToDoctor** – labsupervisor/labadmin; auto-returns to WITH_DOCTOR after lab approval

5.5 ClinicalContract

Handles all clinical updates. Each function validates role and status, then appends the new `visitCID` to `cidHistory[]` with a descriptive reason. Content modification happens off-chain before the chaincode call:

- **UpdateDiagnosisNotes** – doctor only; reason: `DIAGNOSIS_NOTES_UPDATED`
- **RecordVitals** – nurse only; reason: `VITALS_RECORDED`
- **AddCareNote** – nurse only; reason: `CARE_NOTE_ADDED`
- **UpdatePrescription** – doctor only; reason: `PRESCRIPTION_UPDATED`
- **DispenseMedication** – pharmacist only; requires `VISIT_FINALIZED`

5.6 LabContract

Four-stage lab request lifecycle. All content in IPFS; contract validates role/state and records CID transitions:

- **AcknowledgeLabRequest** – labreceptionist/labadmin; records `LAB_ACKNOWLEDGED:<reqId>`
- **SubmitLabResult** – labtechnician/radiologist/labadmin; records `LAB_RESULT_SUBMITTED:<reqId>`
- **ApproveLabResult** – labsupervisor/labadmin; advances visit back to WITH_DOCTOR

5.7 ClaimsContract

Insurance claim lifecycle. Unlike clinical data, claim metadata (`claimId`, `claimAmount`, `claimStatus`, `auditNotes`) is stored **on-chain** because insurance auditors need legally defensible tamper-proof records:

- **SubmitClaim** – billingofficer; requires `RECORD_FINALIZED`; validates positive float amount
- **AuditClaim** – claimsauditor; requires non-empty `auditNotes`; advances to `CLAIM_UNDER_AUDIT`
- **ProcessClaim** – insuranceofficer; decision must be `APPROVED` or `REJECTED`; rejection requires reason

5.8 EhrContract

Manages the persistent patient EHR under `EHR:<patientId>`:

- **InitEHR** – receptionist/admin; called once at registration; creates `cidHistory[]` with first entry
- **UpdateEHRCID** – requires `assertEHRWriteAccess`; doctor/nurse must have patient consent; appends new CID with section granularity and reason
- **GetCurrentCID** – requires `assertReadAccess` with section `ehr`; returns current IPFS CID
- **GetEHRCIDHistory** – full `cidHistory[]` array; input to aggregate EHR verification
- **GetEHRBlockHistory** – full blockchain transaction history of the EHR key

5.9 AccessContract

The consent management contract. Stores under `ACCESS:<patientId>` with three arrays: `grants[]`, `requests[]`, `auditLog[]`.

```
{
  "patientId": "PAT-042",
  "grants": [{
    "grantId": "PAT-042-G1",
    "granteeId": "doctor",
    "granteeRole": "doctor",
    "sections": ["ehr", "visits"],
    "grantedBy": "patientService",
    "grantedAt": "2026-04-19T10:30:00Z",
    "expiresAt": "2026-06-01T00:00:00Z",
    "revoked": false
  }],
  "requests": [...],
  "auditLog": [...]
}
```

Listing 5: On-chain consent record structure

Key functions: **GrantAccess** (patientService/admin only), **RevokeAccess** (instant – sets `revoked: true` on-chain), **RequestAccess** (any staff), **ApproveRequest / RejectRequest** (patientService only), **GetActiveGrants**, **GetAuditLog**, **LogAccess** (records READ events).

6 Individual Contributions

6.1 Avijit Ram – Cryptographic Consent Management with Role Embedding and Aggregate Transactions

6.1.1 Problem Being Solved

Existing healthcare systems enforce access control at the application layer – a middleware check based on a database flag. This is fundamentally bypassable: a compromised API server, a rogue database administrator, or a software bug can circumvent it. There is no cryptographic guarantee that the party invoking a chaincode function actually has the role they claim. Furthermore, patients have no mechanism to verify that their records have not been silently accessed or modified.

6.1.2 Role Embedding in X.509 Certificates

The core mechanism embeds the role string inside the Fabric CA-issued X.509 certificate using the `-id.attrs` flag during enrollment:

```
# Step 1: Register (done by CA admin once)
fabric-ca-client register \
  --id.name nurse \
  --id.secret nursepw \
  --id.type client \
  --id.attrs "role=nurse:ecert" \
  --tls.certfiles "$CADIR/tls-cert.pem" \
  -u https://ca.hospital.example.com:7055

# Step 2: Enroll (done at server startup)
fabric-ca-client enroll \
  -u https://nurse:nursepw@ca.hospital.example.com:7055 \
  --caname ca-hospital \
  --tls.certfiles "$CADIR/tls-cert.pem" \
  -M "$USERS/nurse/msp"
```

Listing 6: Fabric CA enrollment with role attribute embedding

The `:ecert` suffix instructs the CA to embed this attribute in the enrollment certificate as an X.509 extension. The chaincode reads it directly:

```
// In accessControl.js -- called inside chaincode, inside the peer
const role = ctx.clientIdentity.getAttributeValue('role'); // -> "nurse"

// Certificate Subject shows the embedded attribute:
// CN=nurse, OU=client, O=hospital
// X.509v3 Extension - Fabric Custom Attributes:
//   role: nurse
```

Listing 7: Chaincode reads role from certificate – no DB lookup

Security property: Even if the entire API server is compromised, a nurse identity cannot invoke a doctor-only chaincode function. The chaincode reads `role=nurse` from the certificate and rejects the call. The security boundary has moved from the application layer (bypassable) to the cryptographic layer (unforgeable, since forging requires compromising the CA's private key).

6.1.3 The patientService Pattern for Patient Consent

Patients are not enrolled as Fabric identities in the CA. Instead, a single `patientService` identity is enrolled with `role=patientService:ecert`. When a patient authenticates to the patient portal (bcrypt-hashed password in SQLite), the patient-api verifies the JWT and calls chaincode using the `patientService` certificate, passing the `patientId` as an argument.

```
// In AccessContract.GrantAccess -- chaincode
async GrantAccess(ctx, patientId, granteeId, granteeRole, sectionsJson,
    expiresAt) {
  // patientService = patient-api service account
  // (API verified patient's JWT before calling this)
  const { userId } = requireRole(ctx, 'patientService', 'admin');
  // ... validate sections, expiry, create grant ...
}
```

Listing 8: patientService pattern – trust delegation chain

Trust chain: CA trusts patientService identity → chaincode trusts patientService calls → patient-api verifies patient JWT before calling chaincode. Only the patient (via their portal) can modify their own consent records.

6.1.4 Five Security Properties of the Consent System

1. **Identity Authenticity** – role proven by CA-signed X.509 certificate, not a database field
2. **Access Authorization at Chaincode Level** – `assertReadAccess()` runs inside the peer; API server cannot bypass it
3. **Patient Sovereignty** – only `patientService` (after verifying patient JWT off-chain) can invoke consent-modifying functions
4. **Time-bounded and Instantly Revocable** – `expiresAt` enforced inside chaincode; `RevokeAccess()` takes effect immediately with zero delay
5. **Immutable Audit Trail** – every GRANT, REVOKE, REQUEST, READ event permanently recorded in `ACCESS:<patientId>.auditLog[]` on-chain

6.1.5 Aggregate Transaction Verification

The aggregate transaction feature addresses a complementary problem: the actual clinical content lives in IPFS, and a patient cannot easily verify IPFS integrity directly. On-chain visit records accumulate many transactions over time – the patient has no simple way to know if an unexpected transaction was added.

For visit records, the aggregate is computed from blockchain transaction IDs returned by `GetVisitHistory`:

```
// In patient-api/src/routes/signatures.js
function computeAggregate(ids) {
  const real = ids.filter(id => id);
  if (!real.length) return null;
  return crypto.createHash('sha256')
    .update([...real].sort().join(''))  // sort for determinism
    .digest('hex');
}
```

Listing 9: computeAggregate – SHA-256 over sorted txIds

Sorting is critical: the same set of transactions must always produce the same hash regardless of traversal order.

For EHR records, the aggregate is computed over the CIDs in the EHR's `cidHistory[]`. Since IPFS CIDs are themselves SHA-256 content hashes, this produces a hash of hashes – any content modification produces a different CID which produces a different aggregate:

```
function computeEhrAggregate(entries) {
  const cids = entries.map(e => e.cid).filter(Boolean);
  return computeAggregate(cids);
}
```

Listing 10: computeEhrAggregate – hash over IPFS CIDs

Verification workflow:

1. Patient views `GET /signatures` – sees interaction timeline and `aggregateHash`
2. Patient copies/screenshots the hash (e.g., `"e4f7a2b9c1..."`)
3. Later, patient opens `GET /signatures/verify/visit/PAT-001-V1?hash=e4f7a2b9c1...`
4. Backend recomputes aggregate from current blockchain history
5. If `valid: true` – no new transactions added since the hash was saved
6. If `valid: false, txCount: 8` (was 7) – a new unexpected transaction was added

The `GET /signatures` endpoint also infers human-readable action labels and actor identities from state transitions, presenting a timeline of every clinical action to the patient without requiring them to understand raw blockchain data.

6.2 Jenish Shiroya – Hospital Staff Registration with Dual-Store Identity Management

6.2.1 Problem Being Solved

In a blockchain-based system, staff members must have two types of credentials: an application-layer credential (username/password for API login) and a blockchain-layer credential (X.509 certificate for chaincode invocation). Keeping these two stores in sync atomically is a non-trivial engineering problem. If a user is added to the application store but CA enrollment fails, the user can log in but cannot invoke chaincode. Jenish Shiroya's contribution is the design and implementation of the dual-store, atomic, rollback-capable staff registration system.

6.2.2 The Hospital Admin Staff Registration Portal (Staff.jsx)

The `Staff.jsx` React component implements the hospital admin's staff management interface. It connects exclusively to SYS1 (`http://10.162.37.29:3001`) for user management – the machine where peer0 runs and where all Fabric CA enrollment happens. The component shows a staff directory grid with role-appropriate icons and a modal form for creating new staff (role, username, password).

6.2.3 The Registration Endpoint (POST /auth/users)

The endpoint implements a six-phase atomic registration process:

1. **Input validation** – express-validator; admin role check gates entire handler
2. **Duplicate prevention** – checks `users.json` before any operations
3. **Peer mapping** – derives correct peer assignment by role: doctors → `peer1`, nurses/pharmacists/medrecordofficers → `peer2`, others → `peer0`
4. **Application store write** – `bcrypt.hash(password, 10)`; writes to `users.json`
5. **Fabric CA enrollment** – invokes `enrollregisteruser.sh` via `execFile` with 30-second timeout
6. **Rollback on failure** – if enrollment fails, re-reads `users.json`, deletes the new entry, and re-writes to keep both stores in sync

```
// Phase 4: Write to application credential store
USERS[username] = { password: hash, role, mspId: 'HospitalMSP', peer };
fs.writeFileSync(USERS_FILE, JSON.stringify(USERS, null, 2));

// Phase 5: Fabric CA enrollment
execFile('bash', [ENROLL_SCRIPT,
  '--org', 'hospital', '--username', username,
  '--password', password, '--type', 'client', '--role', role
], { timeout: 30000 }, (err, stdout, stderr) => {
  if (err) {
    // Phase 6: Rollback -- keep both stores in sync
    const current = getUsers();
    delete current[username];
    fs.writeFileSync(USERS_FILE, JSON.stringify(current, null, 2));
    return res.status(500).json({ success: false,
      error: 'Fabric CA enrollment failed. User not created.' });
  }
  return res.status(201).json({ success: true,
    data: { username, role, mspId: 'HospitalMSP', peer } });
});
```

Listing 11: Atomic dual-store registration with rollback

6.2.4 MSP Folder Creation via enrollregisteruser.sh

The shell script implements the four-step MSP folder creation:

```
# Step 1: Enroll CA admin into temp home (authorization to register)
```

```
fabric-ca-client enroll -u https://admin:adminpw@$CA_URL \
  --caname $CA_NAME --tls.certfiles $CA_TLS_CERT --home $TEMP_HOME

# Step 2: Register with role attribute embedded in certificate
fabric-ca-client register --caname $CA_NAME \
  --id.name $USERNAME --id.secret $PASSWORD --id.type $ID_TYPE \
  --id.attrs "role=${ROLE_ATTR}:ecert" \
  --tls.certfiles $CA_TLS_CERT --home $TEMP_HOME

# Step 3: Enroll -- fetches signed X.509 cert + private key
fabric-ca-client enroll -u https://$USERNAME:$PASSWORD@$CA_URL \
  --caname $CA_NAME --tls.certfiles $CA_TLS_CERT -M $MSP_DIR

# Step 4: Create config.yaml with NodeOUs for OU-based access control
cat > "$MSP_DIR/config.yaml" <<EOF
NodeOUs:
  Enable: true
  ClientOUIdentifier:
    Certificate: cacerts/${CA_CERT_FILE}
    OrganizationalUnitIdentifier: client
  ...
EOF
```

Listing 12: enrollregisteruser.sh – four-step identity lifecycle

6.2.5 JWT Token Design and Two-Layer Gateway Security

After login, the auth endpoint issues a JWT with `{userId, role, mspId: 'HospitalMSP', peer}`. The `peer` field locks gateway routing for all subsequent chaincode calls to the specific peer that role is mapped to. The `gatewayManager.js` reads this field to select the correct entry from the `PEER_MAP` in `peerRouter.js`.

The two-layer security model: Layer 1 is JWT middleware (`authenticate` + `requireRole`) at the API level. Layer 2 is the certificate-embedded role at the chaincode level. Even if Layer 1 is bypassed (API compromised), Layer 2 rejects the transaction because the certificate's role doesn't match what `requireRole(ctx, ...)` expects in chaincode. Both layers must be bypassed for an unauthorized action to succeed.

6.3 Rahul Yadav – OCR-Assisted Medical Record Ingestion and EHR PDF Export

6.3.1 Problem Being Solved

A patient's medical history doesn't begin when they first use this system. They may have years of records in paper form – old lab reports, discharge summaries, specialist letters – that are inaccessible to the blockchain-based EHR. Rahul Yadav's contribution bridges this offline-to-blockchain gap through an OCR pipeline that digitizes scanned medical documents and ingests the extracted data into the patient's IPFS-backed EHR.

6.3.2 Server-Side OCR Pipeline (patient-api/src/routes/ocr.js)

Four-stage pipeline, operating entirely on in-memory buffers (no temporary files on disk):

```
// Stage 1: multer memoryStorage -- file buffered in RAM, never touches
   disk
```

```
const upload = multer({ storage: multer.memoryStorage() });

// Stage 2: IPFS archival BEFORE OCR -- immutable source audit trail
const { cid: sourceCid } = await uploadFile(req.file.buffer, filename);

// Stage 3: Tesseract.js recognition -- bundled eng.traineddata (~5MB)
const { data: { text } } = await Tesseract.recognize(
  req.file.buffer, 'eng', { logger: m => logger.debug(m) }
);

// Stage 4: Structured attribute extraction via regex patterns
function extractAttributes(text) {
  return {
    name: text.match(/(?:Patient|Name)\s*:\s*(.+)/i)?.[1]?.trim(),
    date: text.match(/\b(\d{1,2}[\/\-]\d{1,2}[\/\-]\d{2,4})\b/)?.[1],
    hospital: text.match(/(?:Hospital|Clinic|Medical Center)\s*[:\-]?\s
        *(.+)/i)?.[1]?.trim(),
    diagnosis: text.match(/(?:Diagnosis|Findings|Impression)\s*:\s*(.+)/
        i)?.[1]?.trim(),
    treatment: text.match(/(?:Treatment|Plan|Recommendation)\s*:\s*(.+)/
        i)?.[1]?.trim(),
    doctor: text.match(/(?:Dr\.|Doctor|Physician)\s*(.+)/i)?.[1]?.trim()
        ,
  };
}
```

Listing 13: OCR pipeline – in-memory, IPFS-first, structured extraction

Response payload: `text` (raw OCR), `attributes` (six extracted fields), `sourceCid` (IPFS CID of source image), `filename`.

6.3.3 EHR Integration – Two-Stage UX and Blockchain Commit

The OCR result flows into the EHR through a deliberate two-stage UX preventing accidental writes of bad OCR output to the immutable ledger:

- **Stage 1 (Upload + Preview):** Patient uploads document, OCR runs, results displayed in editable form alongside extracted attributes. Patient can review and correct before writing to chain.
- **Stage 2 (Save):** On explicit confirmation, `POST /ehr/medical-history` fetches current EHR CID → fetches JSON from IPFS → appends `{text, sourceType: 'ocr', attributes, sourceCid}` to `ehr.medicalHistory[]` → pins updated JSON → calls `EhrContract:UpdateEHRCID` on-chain

The `sourceType: 'ocr'` tag allows any downstream consumer or audit tool to distinguish OCR-ingested entries from manually entered or clinically recorded data. The `sourceCid` permanently ties each EHR entry to its IPFS-archived source image for future verification.

6.3.4 EHR PDF Export (generateEhrPdf)

Client-side PDF generation using `jsPDF + jspdf-autotable`. No server round-trip: EHR data is already in React state (fetched from patient-api), and `generateEhrPdf(ehr, profile)` renders it directly in the browser.

PDF sections as autotable-formatted tables: Personal Details (4-column: Name, DOB, Blood Type, Patient ID), Emergency Contact, Allergies (Substance / Reaction / Severity), Chronic Conditions, Ongoing Medications, Immunizations. Robust null handling with `typeof item === 'string'` fallbacks prevents crashes on partially populated records. Every page stamped with page number and "secured by Blockchain". Saved as `Health_Record_<patientId>.pdf`.

Security property: EHR data fetched from the Fabric/IPFS backend does not pass through any intermediate server during export. The patient downloads their own data directly from the browser.

7 Backend Microservices Architecture

7.1 Service Overview

Service	Port	Roles Served	Key Routes
peer0-api	3001	Receptionist, Admin	/patients, /visits, /auth/users (staff reg)
peer1-api	3002	Doctor	/doctor/visits/:id/diagnosis, /prescription, /ehr, /finalize
peer2-api	3003	Nurse, Pharmacist, MedRecordOfficer	/nurse/vitals, /carenote, /pharmacist/dispense, /records/finalize
patient-api	3005	Patient (patientService cert)	/auth, /ehr, /visits, /access, /signatures, /ocr
extorg-api	3004	Lab + Provider/Billing	/lab, /claims
ipfs-service	3006	Internal (all backends)	/pin, /fetch/:cid, /ehr/init, /visit/init

7.2 Gateway Manager and Peer Router

`gatewayManager.js` maintains a connection pool of Fabric Gateway connections, one per role. At startup, it reads each role's MSP directory, creates a gRPC client over mutual TLS, loads the X.509 identity and private key, and establishes a Gateway connection to the `ehrchannel` on the `ehr` chaincode.

`peerRouter.js` defines the `PEER_MAP`: a mapping of role → {peer address, TLS cert path, MSP path, MSP ID}. For peer0-api, this is receptionist and hospitaladmin both pointing to peer0 at `localhost:7051`. The `peerContext` middleware reads the role from the JWT and calls `getContract(role)` to attach the correct contract handle to `req.contract` for all downstream route handlers.

7.3 IPFS Service (ipfs-service)

Wraps the Kubo HTTP API (`http://localhost:5001`). All backends communicate with IPFS exclusively through this service. Key implementation details:

- `POST /pin` – wraps JSON in `FormData`, posts to `/api/v0/add?pin=true&cid-version=1`; returns CIDv1 (base32-encoded)

- `GET /fetch/:cid` – posts to `/api/v0/cat?arg=<cid>`; parses response as JSON
- `POST /ehr/init` – creates `emptyEHR(patientId, demographics)` and pins it; returns CID
- `POST /visit/init` – creates `emptyVisit(...)` with initial `VISIT_OPENED` forwardingLog entry; pins it

8 Network Deployment and Configuration

8.1 Fabric CA Infrastructure

CA	Port	Identities Issued
ca-orderer	7054	Orderer admin, orderer node
ca-hospital	7055	receptionist, doctor, nurse, pharmacist, medrecordofficer, admin, patientService
ca-diagnostics	7056	labreceptionist, labtechnician, radiologist, labsupervisor, labadmin
ca-provider	7057	billingofficer, claimsauditor, insuranceofficer, provideradmin

8.2 Channel Creation (Fabric 2.3+ OSN Admin API)

```
# Step 1: Generate genesis block
configtxgen -profile EHRChannel -outputBlock ./channel-artifacts/
    ehrchannel.block \
  -channelID ehrchannel

# Step 2: Create channel via OSN Admin API (mutual TLS)
osnadmin channel join \
  --channelID ehrchannel \
  --config-block ./channel-artifacts/ehrchannel.block \
  -o localhost:7053 \
  --ca-file "$ORDERER_CA" \
  --client-cert "$ADMIN_TLS_CERT" \
  --client-key "$ADMIN_TLS_KEY"

# Step 3: All 5 peers join the channel
peer channel join -b ./channel-artifacts/ehrchannel.block \
  --tls --cafile "$ORDERER_CA"

# Step 4: Set anchor peers per org for cross-org gossip
peer channel update -o localhost:7050 -c ehrchannel \
  -f ./channel-artifacts/${CORE_PEER_LOCALMSPID}_anchors.tx --tls
```

Listing 14: Channel creation via OSN Admin API

8.3 Chaincode Deployment (Fabric 2.x Lifecycle)

```
# Step 1: Package
peer lifecycle chaincode package ehr.tar.gz \
```

```
  --path ./ehr-chaincode-v3 --lang node --label ehr_1.0

# Step 2: Install on all 5 peers
peer lifecycle chaincode install ehr.tar.gz  # repeat per peer

# Step 3: Approve per org (majority endorsement policy)
peer lifecycle chaincode approveformyorg \
  -o localhost:7050 --channelID ehrchannel --name ehr \
  --version 1.0 --package-id $CC_PACKAGE_ID --sequence 1 --tls

# Step 4: Check readiness
peer lifecycle chaincode checkcommitreadiness \
  --channelID ehrchannel --name ehr --version 1.0 --sequence 1

# Step 5: Commit to channel
peer lifecycle chaincode commit \
  -o localhost:7050 --channelID ehrchannel --name ehr \
  --version 1.0 --sequence 1 --tls \
  --peerAddresses localhost:7051 --peerAddresses localhost:9051 \
  --peerAddresses localhost:8051 --peerAddresses localhost:11051
```

Listing 15: Chaincode lifecycle – multi-org approval required

9 Complete Patient Journey – End-to-End Flow

1. **Patient Registration (peer0-api, receptionist):** Receptionist submits registration → peer0-api calls `ipfs-service POST /ehr/init` → IPFS pins empty EHR JSON → `PatientContract:RegisterPatient` with `ehrCID` → `EhrContract:InitEHR`
2. **Patient Account Creation (patient-api):** Patient registers at portal with patientId + password → bcrypt hash written to SQLite → JWT issued
3. **Open Visit (peer0-api):** peer0-api derives visitId from patient's `visitCount+1` → calls `ipfs-service POST /visit/init` → `VisitContract:OpenVisit` with `visitCID`
4. **Doctor Assignment (peer0-api):** Fetch visit JSON from IPFS → append `DOCTOR_ASSIGNED` to `forwardingLog` → repin → `VisitContract:AssignDoctor` → status WITH_DOCTOR
5. **Doctor Clinical Work (peer1-api):** Update diagnosis/prescriptions in IPFS JSON → `ClinicalContract:UpdateDiagnosisNotes` / `UpdatePrescription`; Read EHR requires patient consent grant; Forward to nurse via `ForwardContract:ForwardToNurse`
6. **Patient Consent Management (patient-api, parallel):** Patient views pending access requests → approves with optional expiry → `AccessContract:ApproveRequest` → grant on-chain → doctor/nurse can now read EHR; Patient can revoke instantly
7. **Nurse Clinical Work (peer2-api):** Record vitals → `ClinicalContract:RecordVitals`; Add care notes → `AddCareNote`; Forward back to doctor → `ForwardContract:ForwardToDocto`
8. **Lab Workflow (extorg-api, DiagnosticsMSP):** Doctor forwards to lab → lab request in IPFS JSON → `ForwardContract:ForwardToLab`; Lab receptionist acknowledges → technician submits results → supervisor approves → `LabResultsBackToDoctor`

9. **Visit Finalization:** Doctor sets finalDiagnosis → `VisitContract:FinalizeVisit` → `VISIT_FINALIZED`
10. **Pharmacy and Medical Records (peer2-api):** Pharmacist dispenses medication → `ClinicalContract` MedRecordOfficer finalizes → `VisitContract:FinalizeRecord` → `RECORD_FINALIZED`
11. **Insurance Claims (extorg-api, ProviderMSP):** Billing officer submits claim → `ClaimsContract:Sub` Auditor reviews → `AuditClaim`; Insurance officer processes → `ProcessClaim`
12. **Discharge (peer0-api, admin):** `VisitContract:DischargePatient` → `DISCHARGED`
13. **Patient Integrity Verification (any time):** Patient opens signatures page → sees interaction timeline and aggregate hash → copies hash → can verify at any future time that no unexpected transactions were added

10 Results and Output

10.1 Network Deployment Results

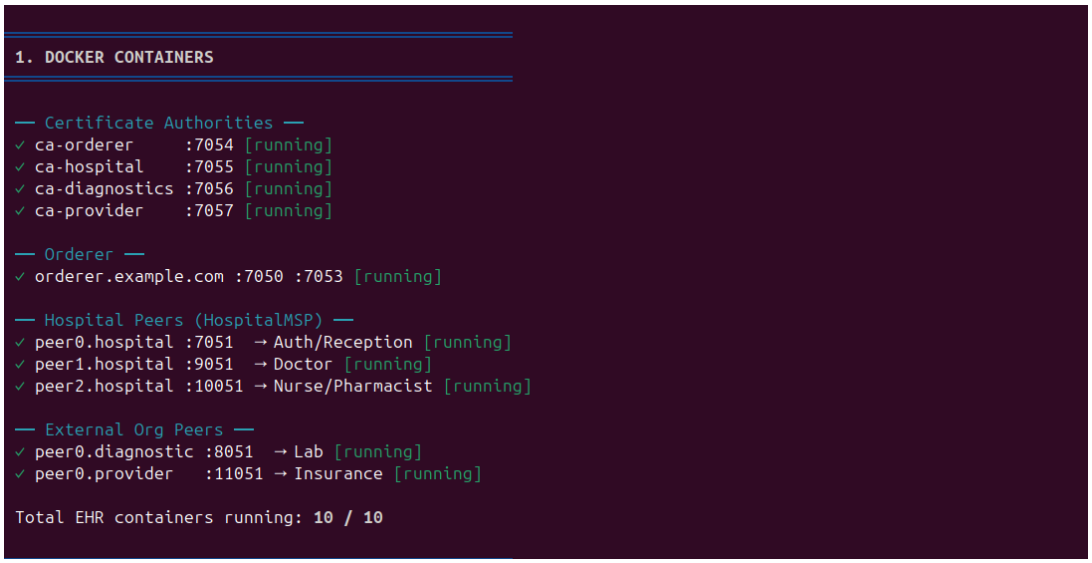

Figure 1: Terminal output of the `status.sh` script showing all 12 Docker containers running healthy: docker-peer0.hospital, docker-peer1.hospital, docker-peer2.hospital, docker-orderer, docker-peer0.provider, docker-peer0.diagnostic, ipfs.ehr.local (healthy), docker-ca-hospital, docker-ca-orderer, docker-ca-provider, docker-ca-diagnostics – all Up 30–44 seconds. Channel: ehrchannel with all 5 peers listed with port assignments and role annotations. IPFS Node running at localhost:5001 (API) and localhost:8090 (Gateway).

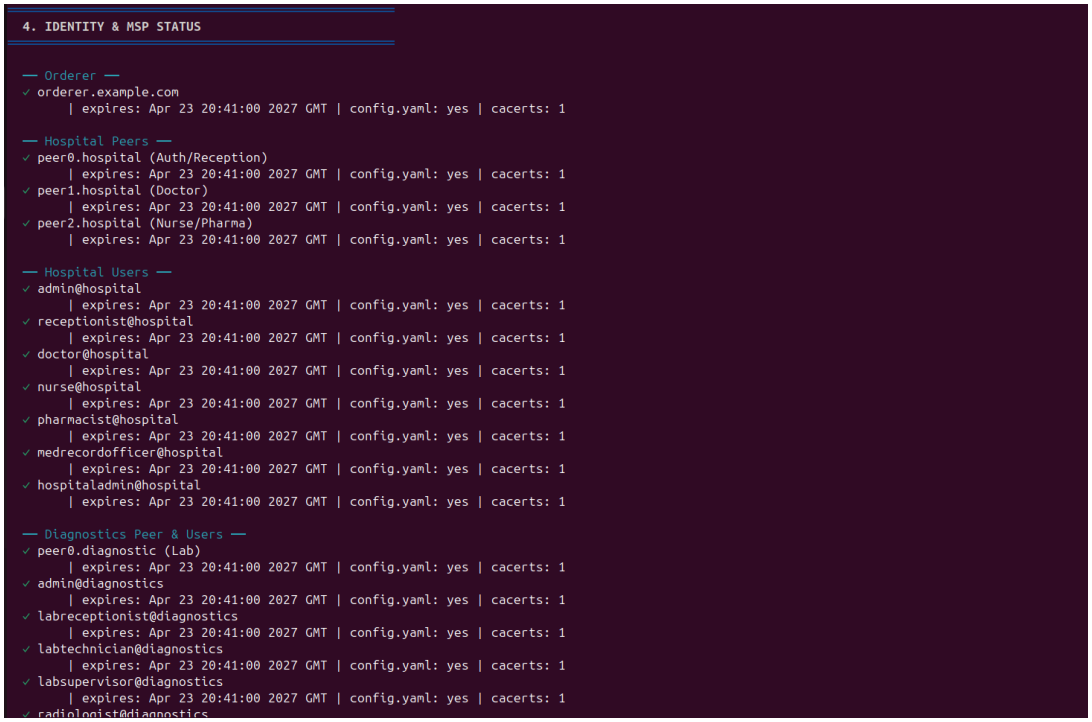

Figure 2: Terminal output showing "ALL IDENTITIES ENROLLED SUCCESSFULLY". Lists all hospital peers with port-to-role mappings and all enrolled identities with their certificate-embedded roles: receptionist→role=receptionist, doctor→role=doctor, nurse→role=nurse, pharmacist→role=pharmacist, hospitaladmin→role=admin, patientService→role=patientService, plus all lab and provider roles.

10.2 On-Chain Ledger State

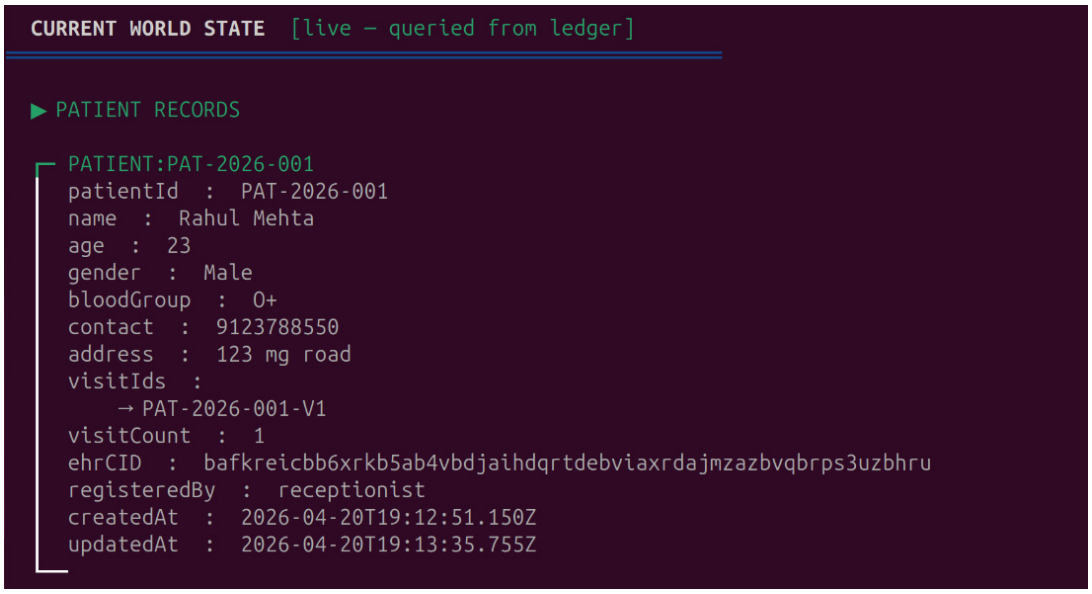

Figure 3: Terminal display of the live ledger world state showing `PATIENT:PAT-2026-001` with all fields: patientId, name (Rahul Mehta), age (23), gender (Male), bloodGroup (O+), visitIds ([PAT-2026-001-V1]), visitCount (1), ehrCID (long bafkrei… IPFS CID), registeredBy (receptionist), createdAt and updatedAt timestamps.

Figure 4: Terminal showing `EHR:PAT-2026-001` with currentCID and full cidHistory with 5 entries: EHR_INITIALISED (receptionist), allergies update (doctor), medicalHistory update (doctor), lifestyle update (nurse), patient contact update (patientService). Each entry shows CID, updatedBy, updatedAt, reason, and section.

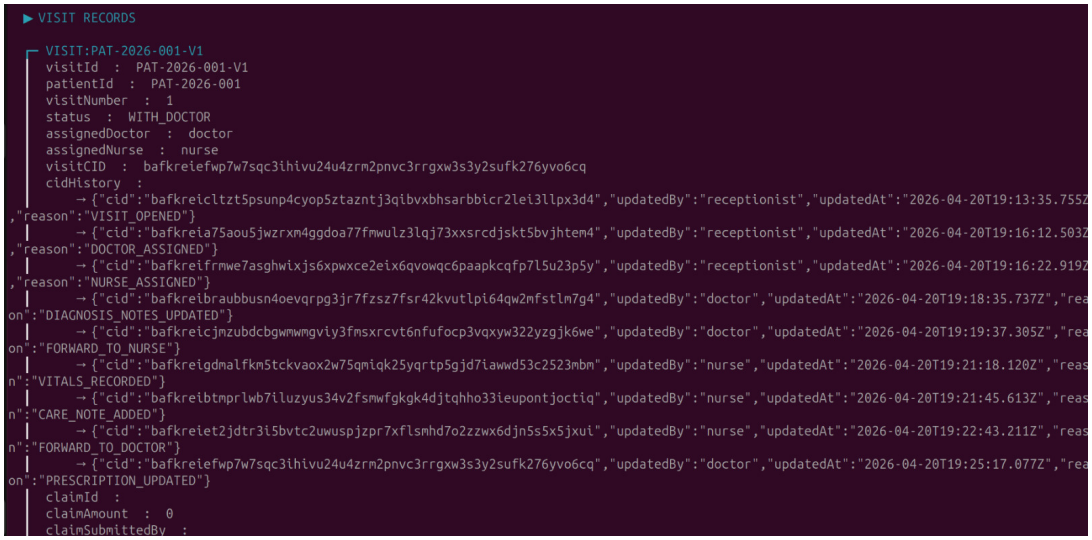

Figure 5: Visit record showing status WITH_DOCTOR, assignedDoctor, assignedNurse, visitCID, and cidHistory with 8 entries: VISIT_OPENED, DOCTOR_ASSIGNED, NURSE_ASSIGNED, DIAGNOSIS_NOTES_UPDATED, FORWARD_TO_NURSE, VITALS_RECORDED, CARE_NOTE_ADDED, FORWARD_TO_DOCTOR, PRESCRIPTION_UPDATED.

10.3 IPFS Content

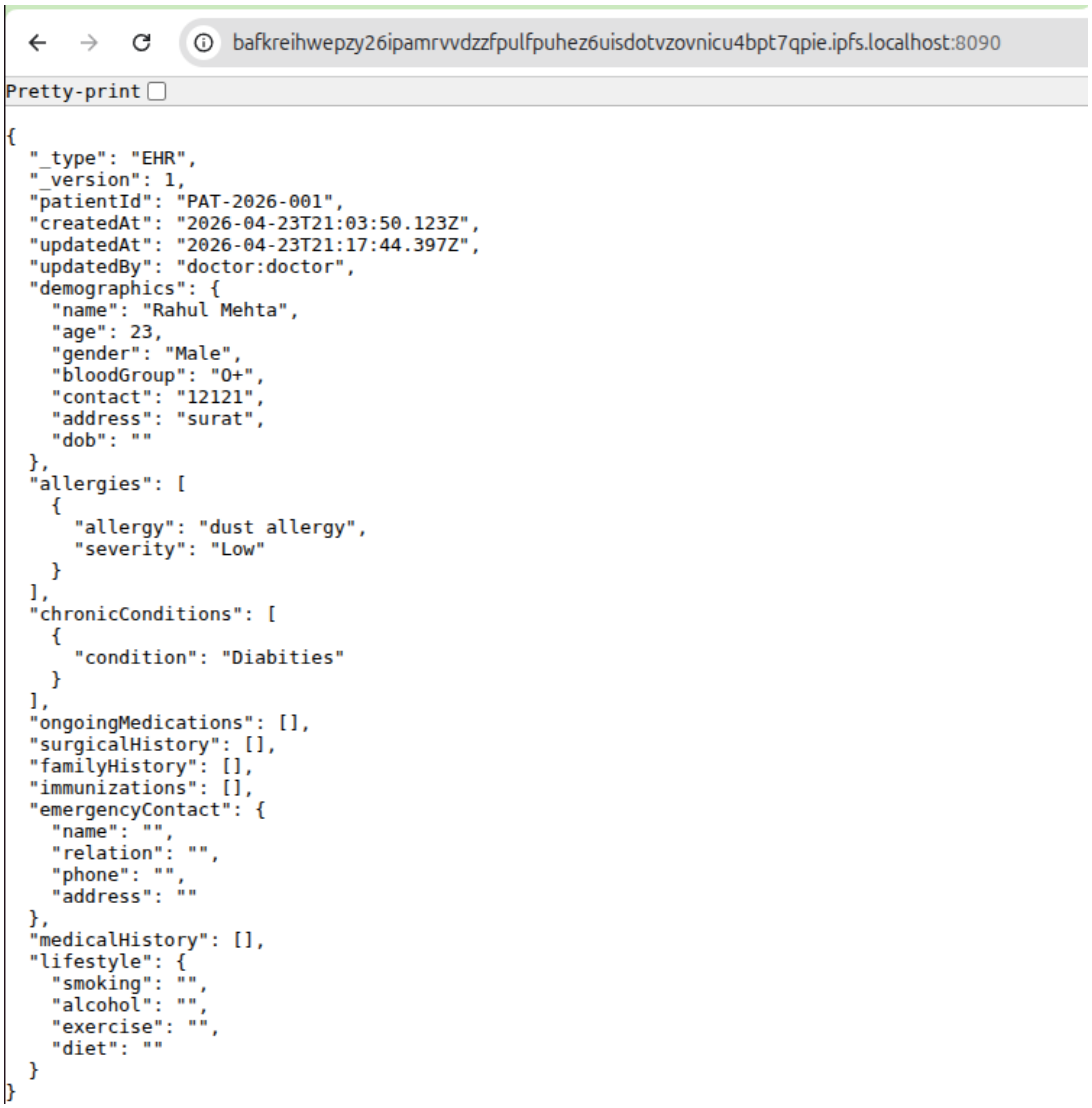

Figure 6: Browser view of the EHR JSON document at an IPFS CID gateway URL (bafkreib25kdosdar. . . .ipfs.localhost:8090). Shows full EHR JSON for PAT-2026-001 with demographics, allergies array (Aspirin, severity High), empty sections, emergencyContact, and a medicalHistory array containing both a manually entered Dengue Fever entry and an OCR-extracted lab report entry with raw Tesseract text and sourceCid.

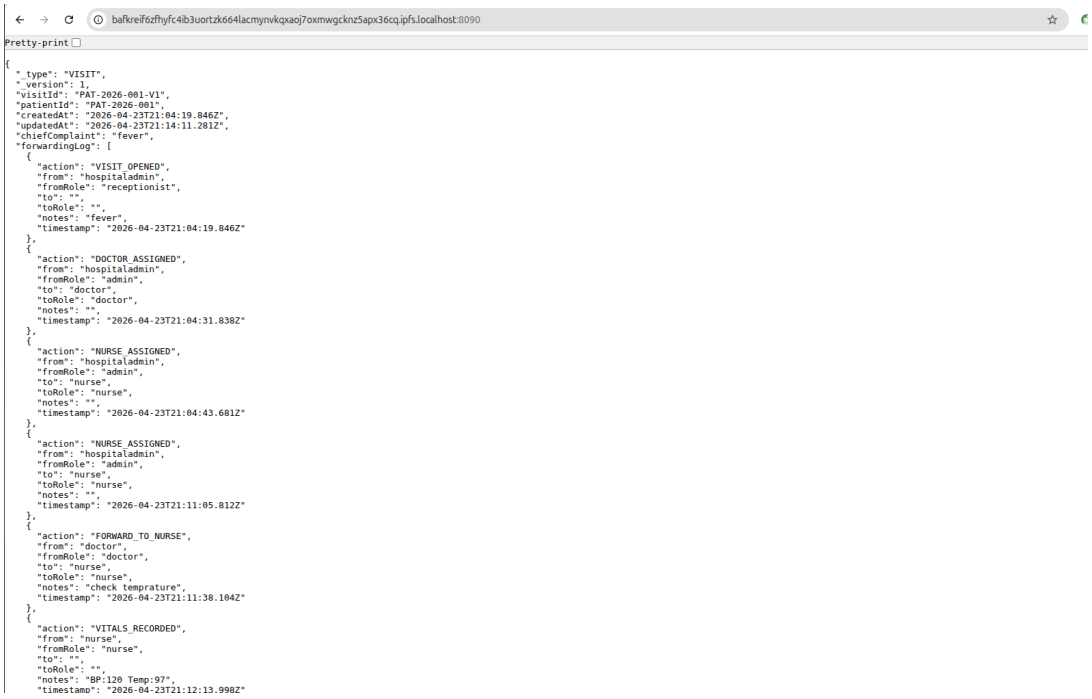

Figure 7: Browser view of Visit JSON showing visitId PAT-2026-001-V1, chiefComplaint "Persistent fever for 3 days with severe headache and body aches", and the forwardingLog array with timestamped entries for every handoff: VISIT_OPENED, DOCTOR_ASSIGNED, NURSE_ASSIGNED, DIAGNOSIS_NOTES_UPDATED, FORWARD_TO_NURSE, VITALS_RECORDED, CARE_NOTE_ADDED, FORWARD_TO_DOCTOR, PRESCRIPTION_UPDATED.

10.4 Blockchain Explorer Output

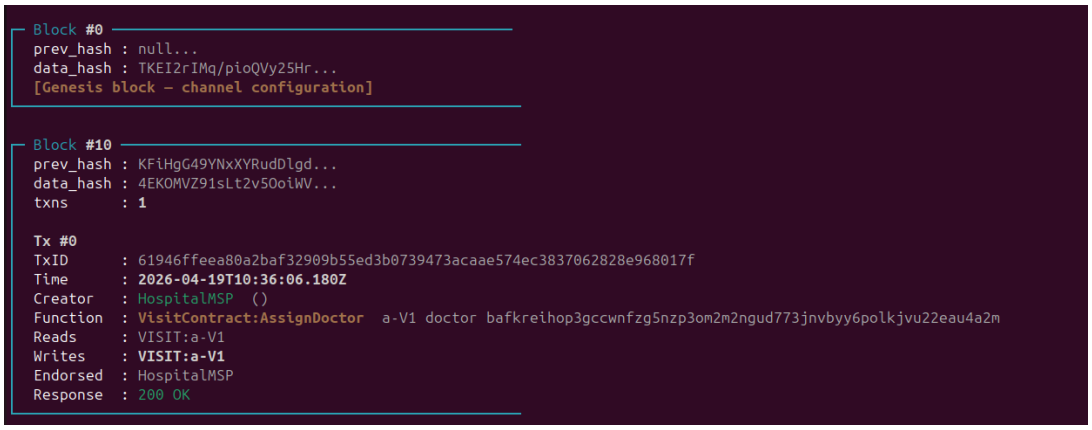

Figure 8: Terminal showing Block #0 (genesis) and Block #10 with prev_hash, data_hash. Block #10 contains Tx #0 with full TxID hash, timestamp 2026-04-19, Creator HospitalMSP, Function VisitContract:AssignDoctor, Reads and Writes on VISIT:a-V1, Endorsed by HospitalMSP, Response 200 OK.

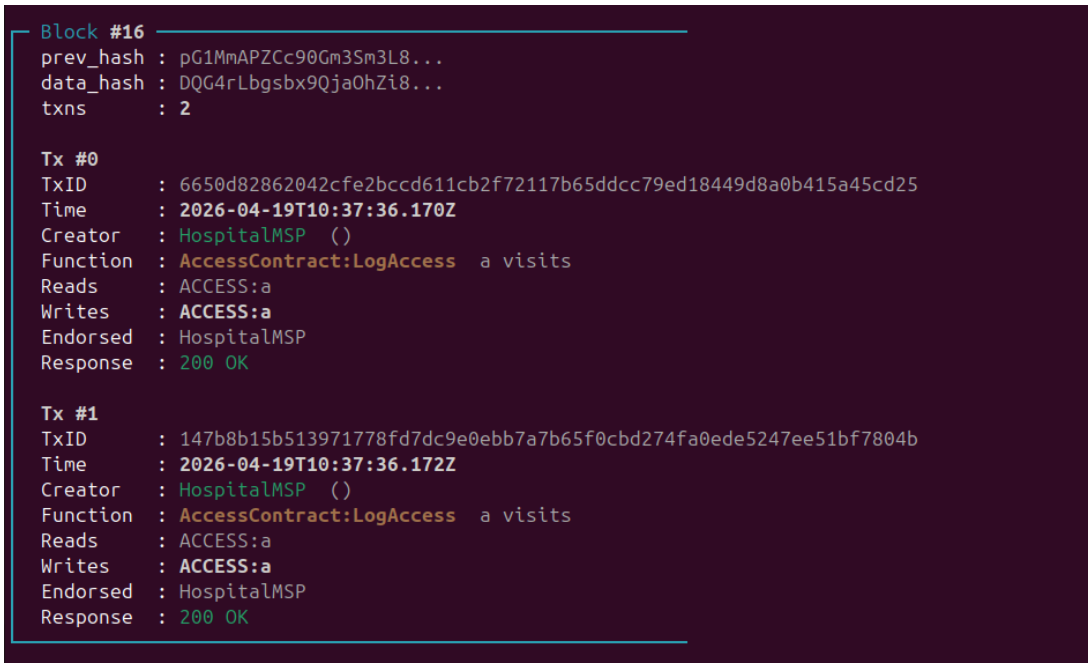

Figure 9: Block #16 containing 2 transactions – both AccessContract:LogAccess calls on ACCESS:PAT-2026-001 from HospitalMSP, demonstrating the on-chain audit logging of patient data access events with full txIDs and timestamps.

10.5 Hospital Portal Screenshots

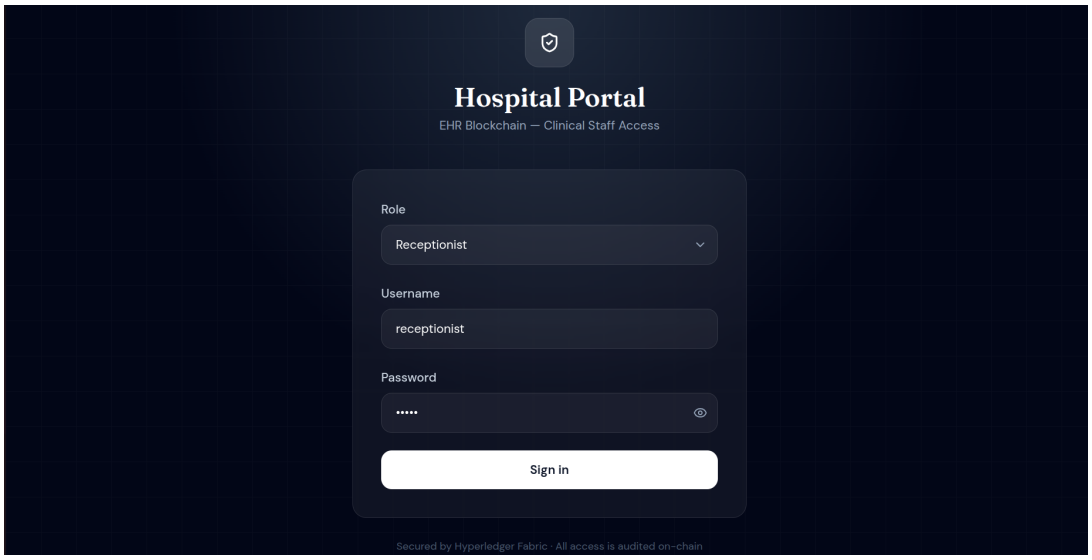

Figure 10: Login page with role selector, username, and password fields on a slate background.

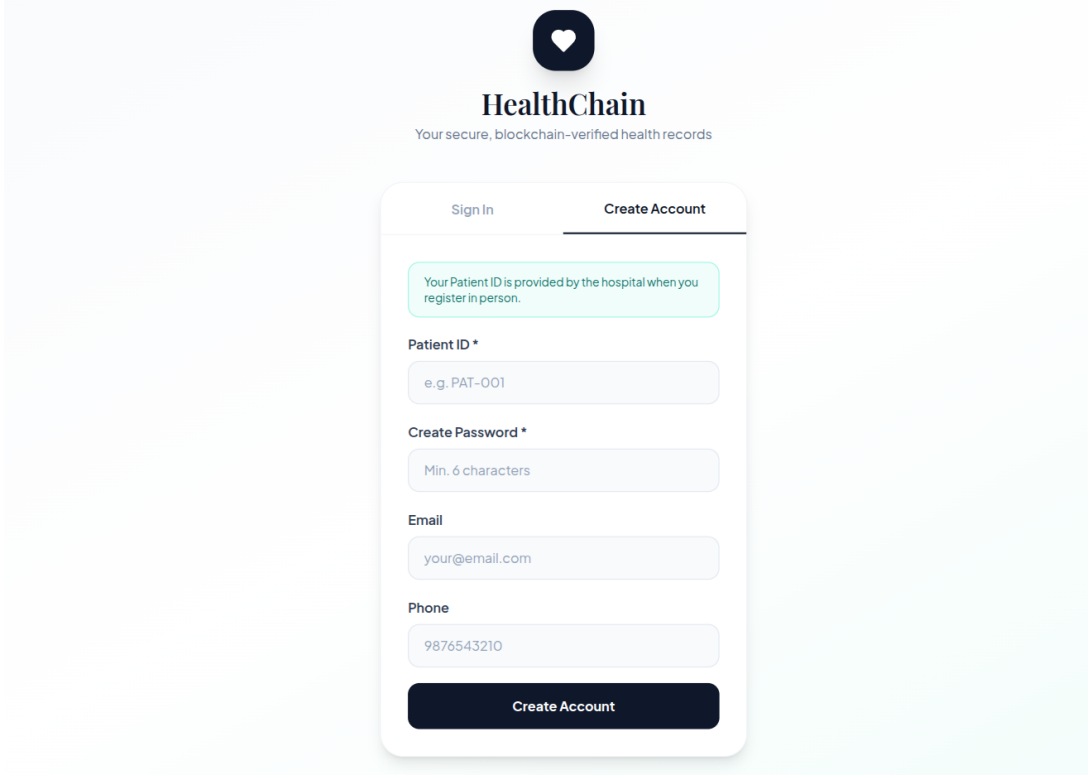

Figure 11: Patient registration form with all required fields and the patient directory listing below.

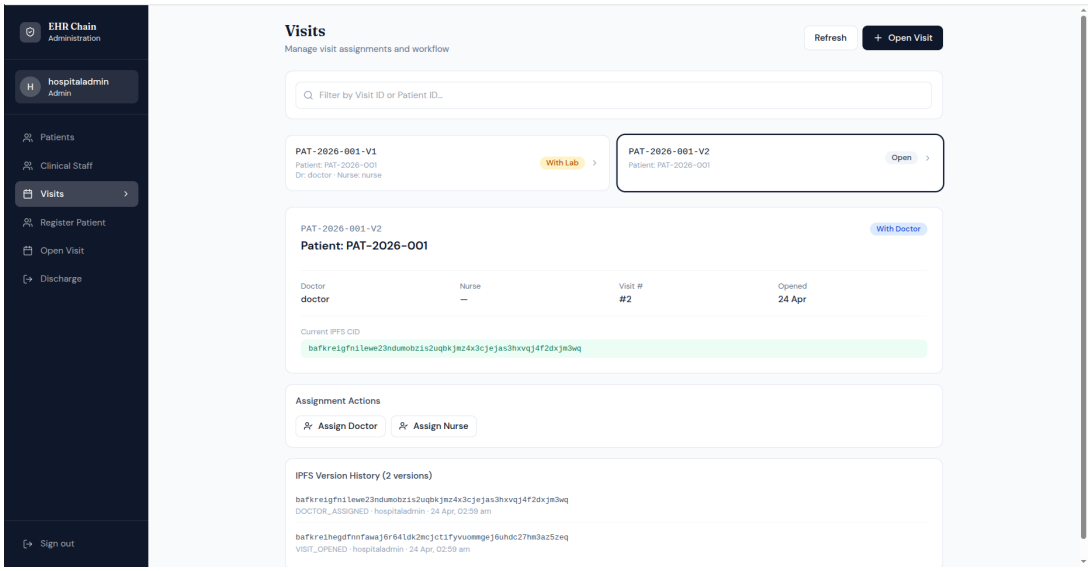

Figure 12: Visit table with status badges (OPEN, WITH_DOCTOR, DISCHARGED), assigned staff, and action buttons.

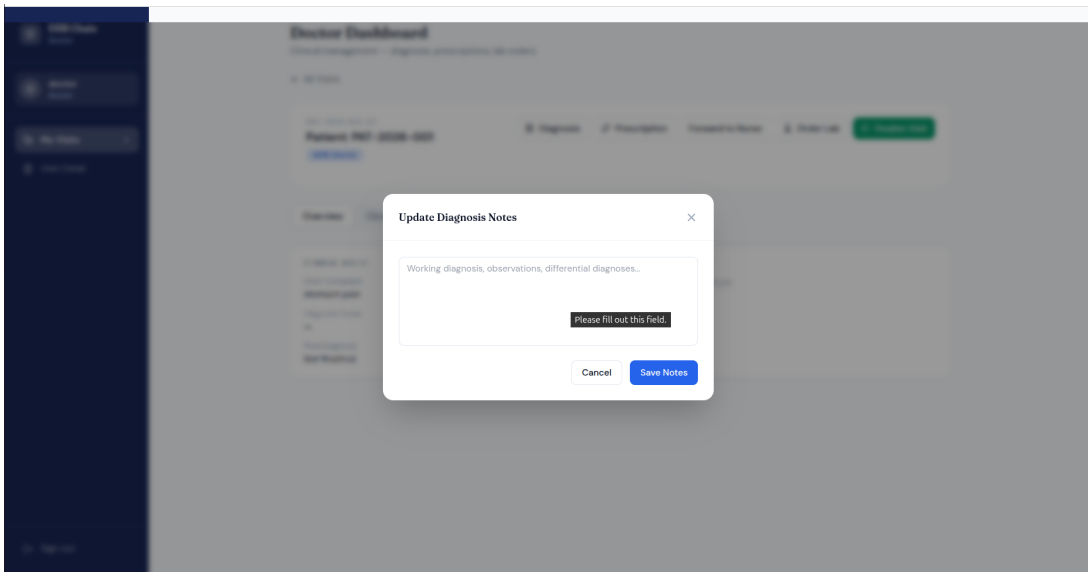

Figure 13: Doctor's visit detail with forwardingLog timeline and open modals for diagnosis and prescription.

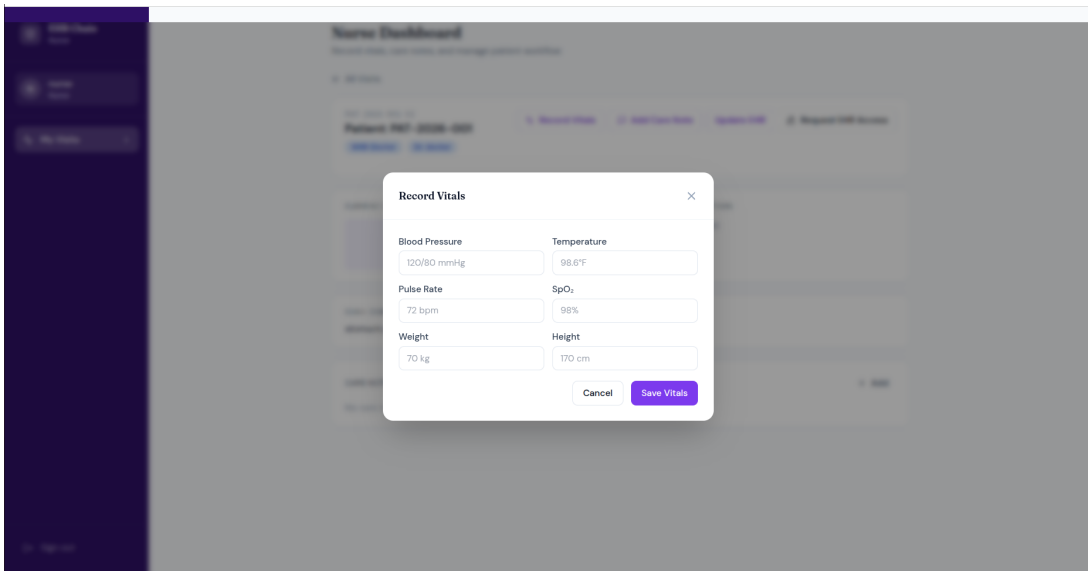

Figure 14: Nurse's vitals recording form (BP, Temperature, Pulse, SpO2, Weight, Height) and care notes section.

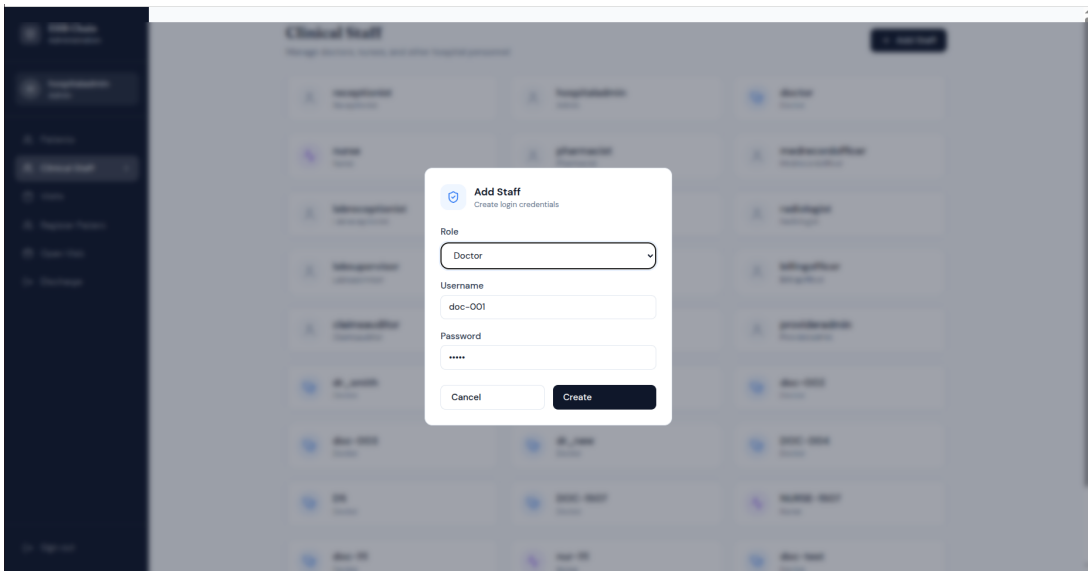

Figure 15: Add Staff modal open with role dropdown and credential fields.

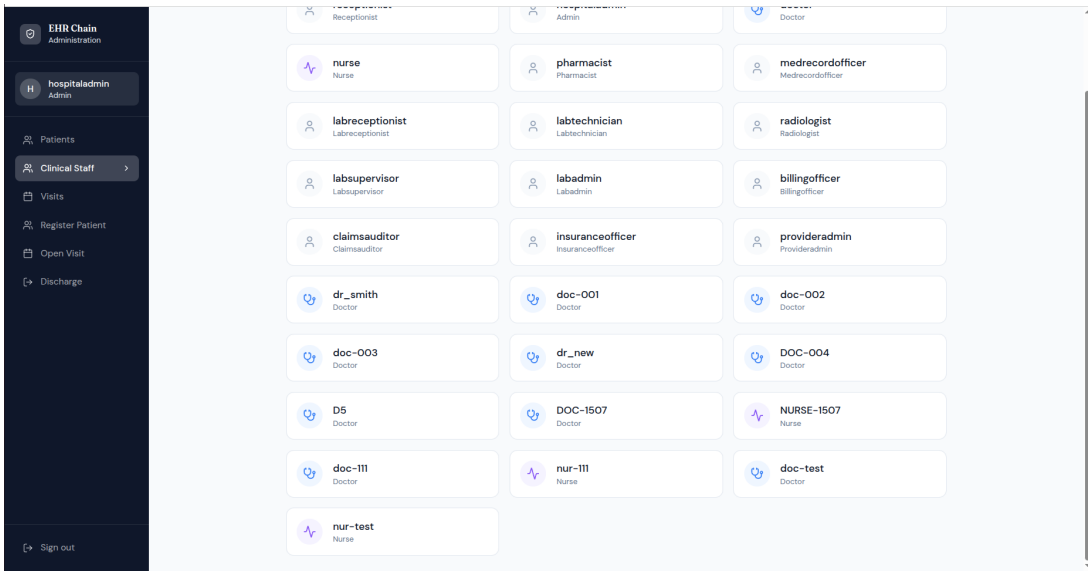

Figure 16: Staff directory with role icons and Add Staff modal open with role dropdown and credential fields.

10.6 Patient Portal Screenshots

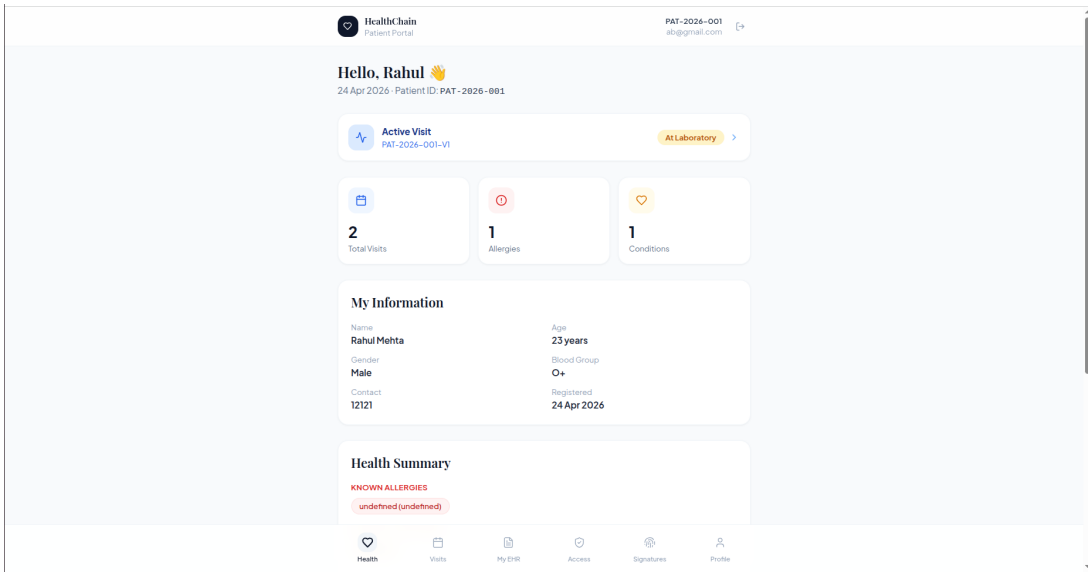

Figure 17: Patient dashboard with welcome message, Patient ID, active visit alert (blue card), and summary cards for allergies and chronic conditions.

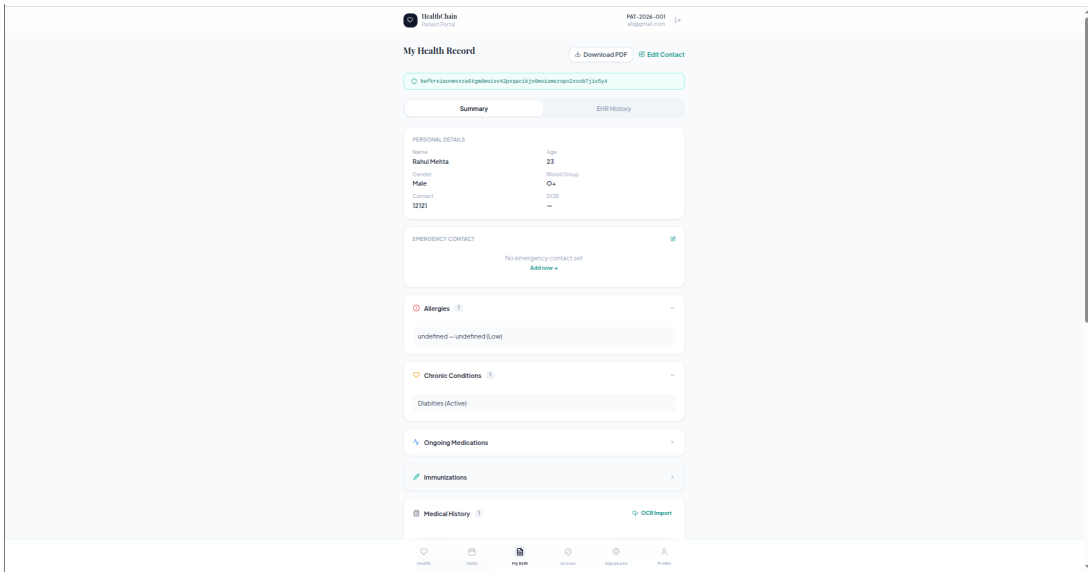

Figure 18: EHR page showing teal IPFS CID badge at top, tabs for Summary and EHR History, demographics grid, allergies table, and emergency contact card.

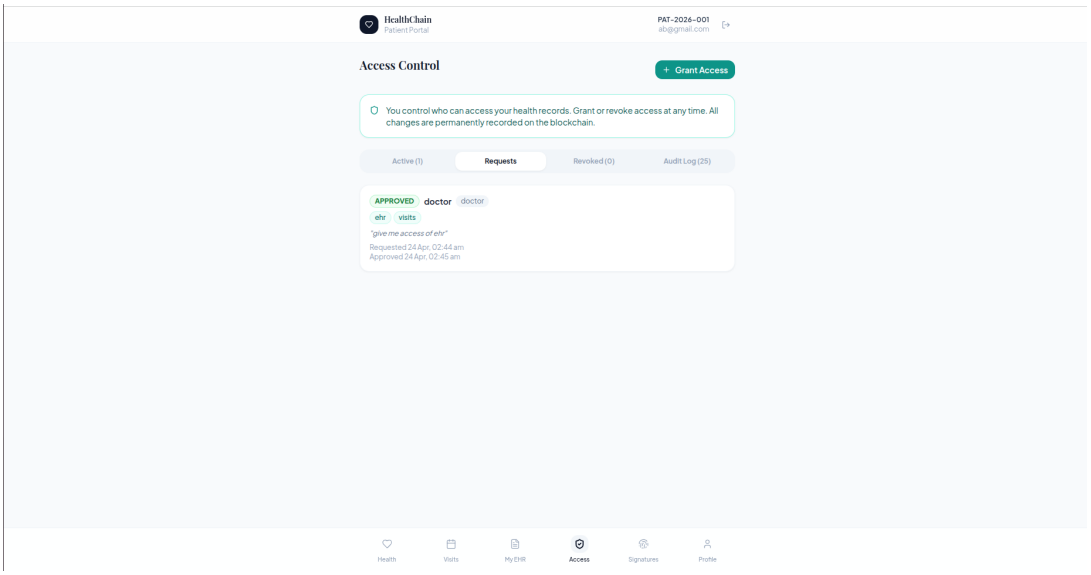

Figure 19: Active Grants list (doctor with sections ehr+visits, expiry, Revoke button), Pending Access Requests (nurse requesting ehr+prescriptions with Approve/Reject buttons), and full Audit Log with GRANT, REVOKE, REQUEST, READ events.

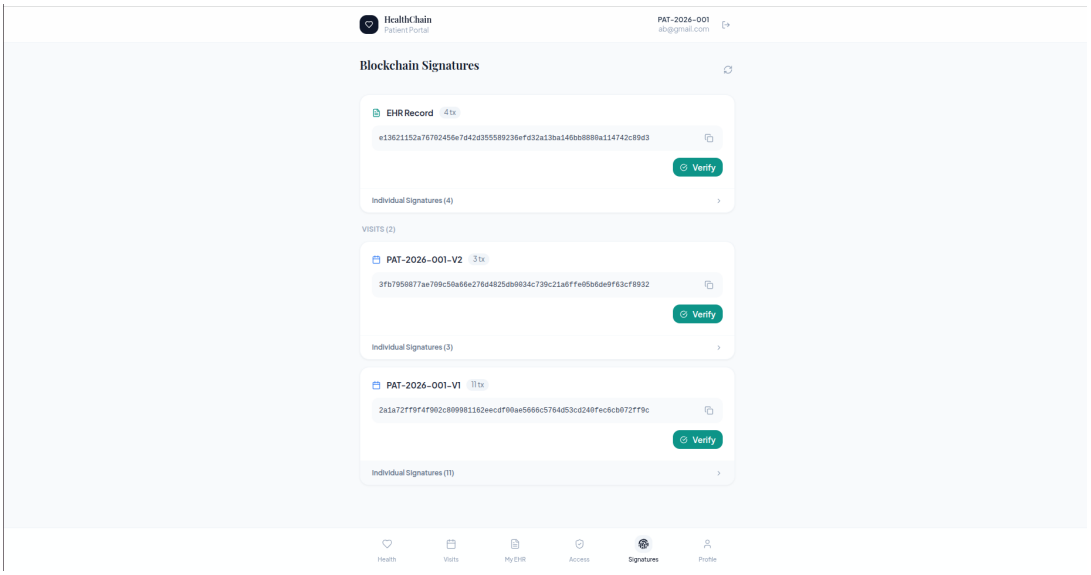

Figure 20: Signatures page showing EHR aggregate hash (64-char hex), each visit with interaction timeline (action labels, actor, timestamp, truncated txID), and Verify button.

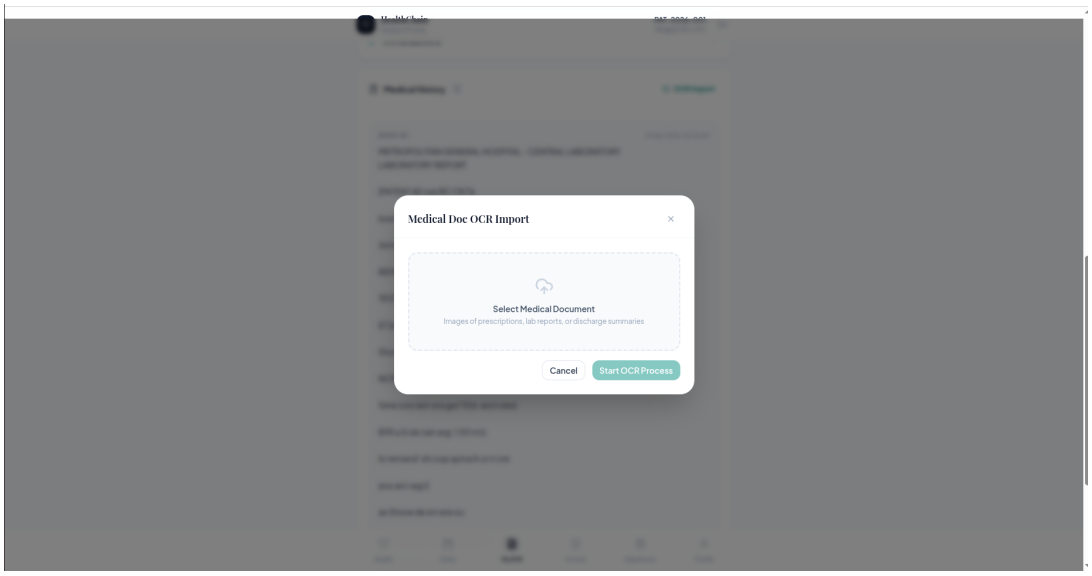

Figure 21: OCR modal showing Stage 1 (file upload dropzone).

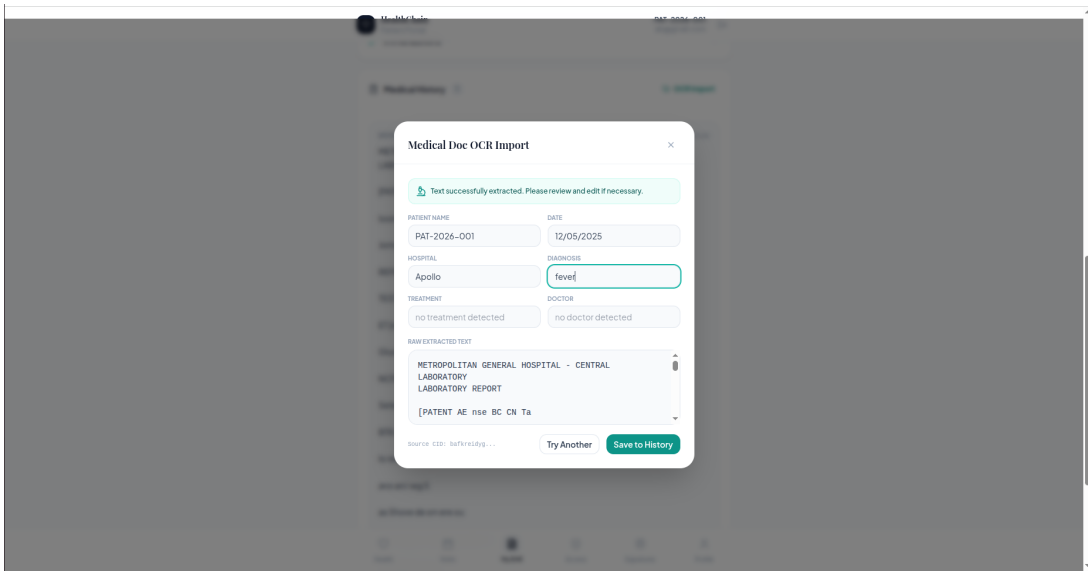

Figure 22: OCR modal Stage 2 showing extracted attributes in editable form alongside raw OCR text preview, with Save to EHR and Discard buttons.

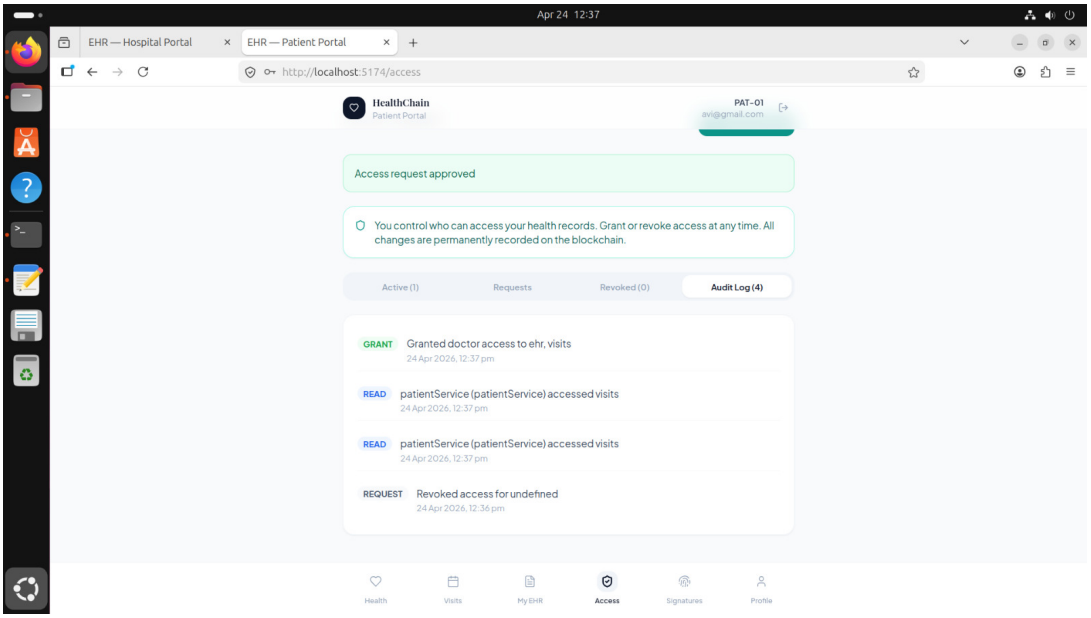

Figure 23: audit log of access control.

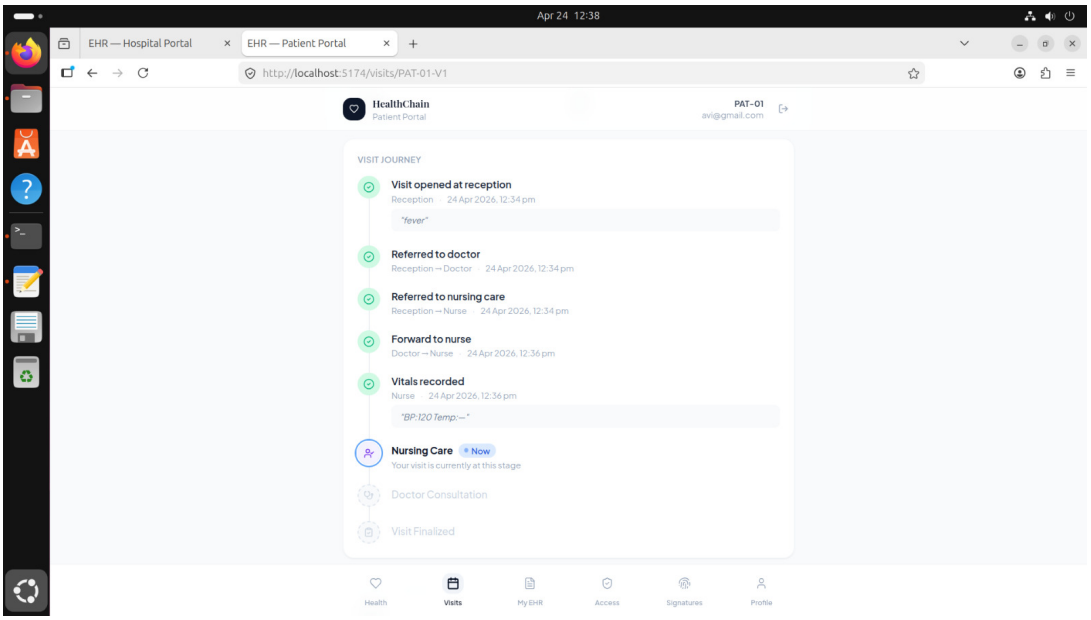

Figure 24: This figure show the patient journey.

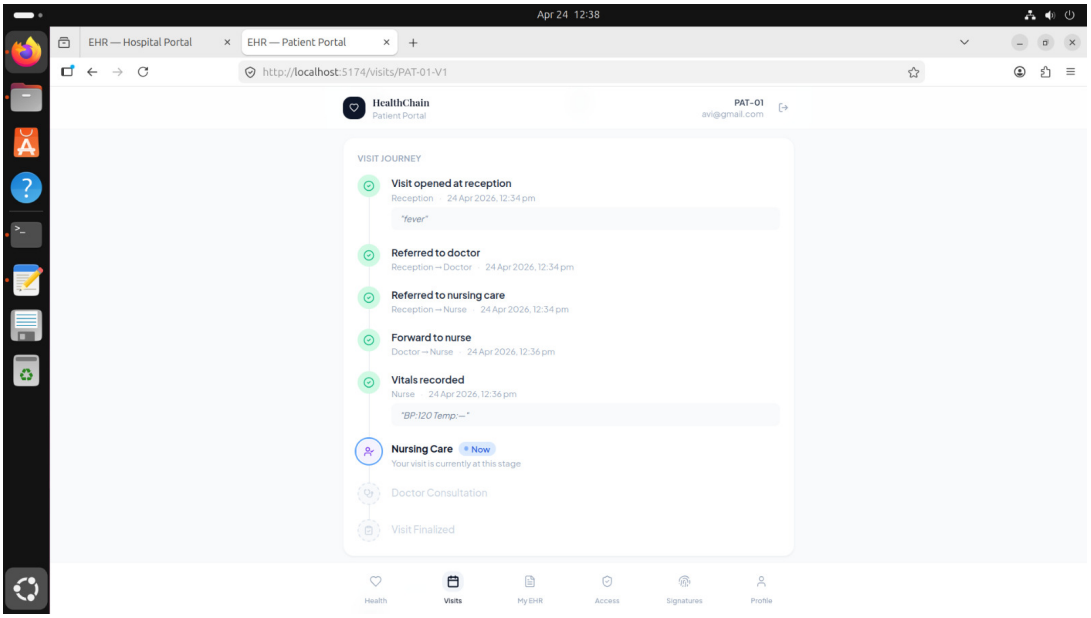

Figure 25: This figure show the patient journey.

11 Technology Stack Summary

Layer	Technology	Version / Details
Blockchain	Hyperledger Fabric	2.5
Consensus	Raft CFT	Single orderer
Smart Contracts	Node.js + fabric-contract-api	v2.x, 8 contracts
Off-chain Storage	IPFS via Kubo	CIDv1
Backend APIs	Node.js + Express	6 microservices
Authentication	JWT + bcryptjs	8h expiry
Fabric Identity	X.509 + Fabric CA	Role attributes :ecert
Patient Auth DB	better-sqlite3	WAL mode
Frontend	React + Vite + Tailwind	2 portals
HTTP Client	Axios	All frontend calls
Validation	express-validator	All routes
OCR	Tesseract.js	English model
PDF Export	jsPDF + jspdf-autotable	Client-side only
Containerization	Docker + Docker Compose	Per-machine files
Network Config	configtx.yaml + OSN Admin API	Fabric 2.3+

12 Conclusion

This project demonstrates a complete, production-architecture decentralized EHR system with three novel contributions from the Hospital Organization subgroup. The cryptographic consent management system moves access control from the bypassable application layer to the cryptographic layer, making it enforceable by the blockchain's own consensus mechanism. The aggregate transaction verification gives patients a human-meaningful integrity tool. The OCR ingestion pipeline bridges the gap between paper medical history and blockchain-anchored EHR.

Together with the overall hybrid Fabric + IPFS architecture, the dual-store atomic staff registration, and the peer-wise role separation, this project represents a comprehensive engineering exploration of what a genuinely secure, patient-empowering, and scalable decentralized health record system looks like in practice.

12.1 Lessons Learned

1. Role embedding in X.509 certificates is far stronger than application-layer RBAC – it shifts the trust boundary to cryptographic infrastructure
2. The hybrid on-chain/off-chain split (CID anchoring) provides both scalability and integrity without compromise
3. Atomic dual-store registration with rollback is essential in systems where application credentials and blockchain identities must stay synchronized
4. Patient-friendly abstractions (aggregate hash, timeline UI) are necessary to make blockchain guarantees accessible to non-technical users
5. The patientService pattern elegantly solves the CA scalability problem for patient identities while maintaining full consent sovereignty

12.2 Impact and Benefits

- **Security:** Role identity enforced at the cryptographic layer – cannot be bypassed by API compromise
- **Patient Sovereignty:** Patients control access to their own medical history with section-level granularity and instant revocation
- **Transparency:** Immutable three-layer audit trail (blockchain TX history, CID version history, consent audit log)
- **Scalability:** Ledger stays lean (only CIDs and small metadata) while clinical content scales freely in IPFS
- **Verifiability:** Aggregate transaction hash lets patients verify record integrity without understanding blockchain internals
- **Interoperability:** OCR ingestion bridges paper historical records into the digital blockchain-backed EHR